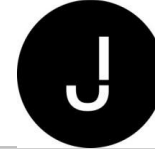


JAVACREAM

*Training
Consulting
Projectmanagement*

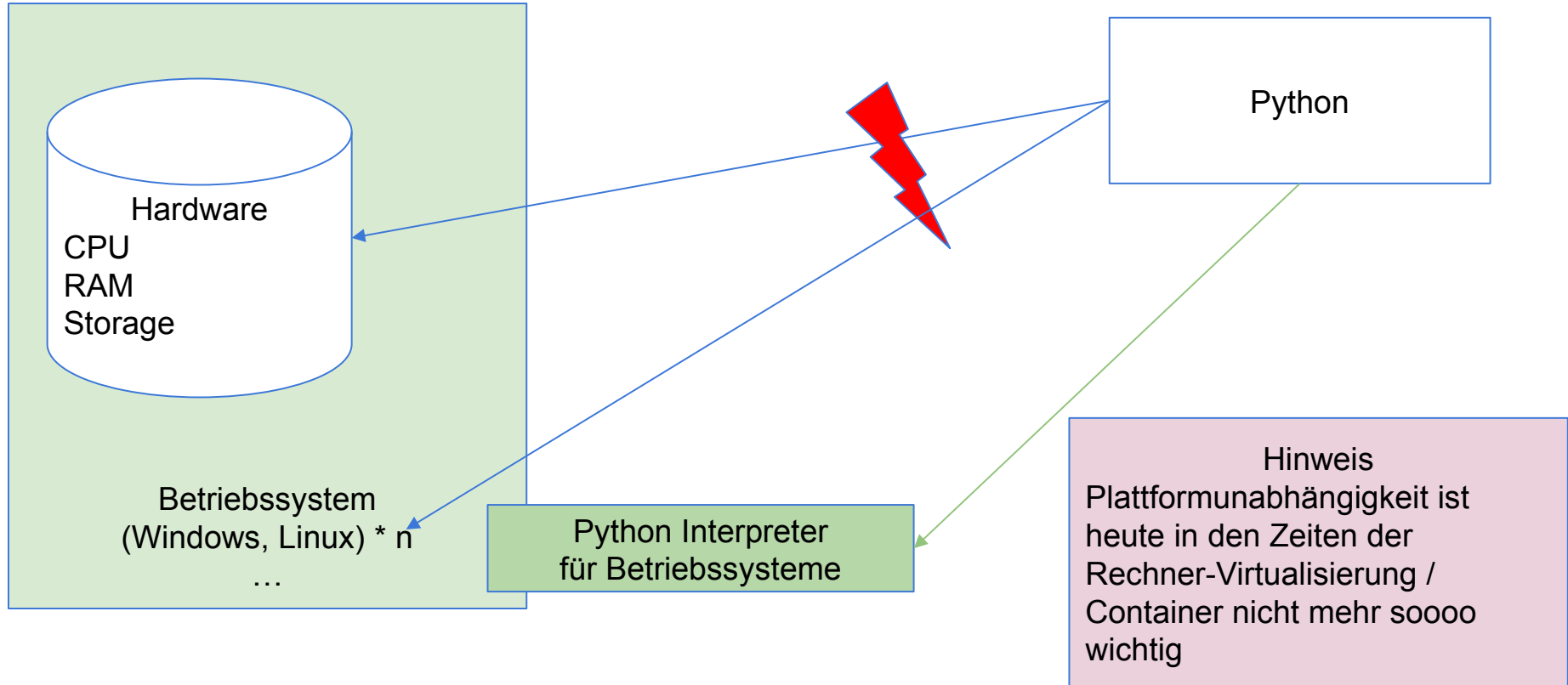
Python

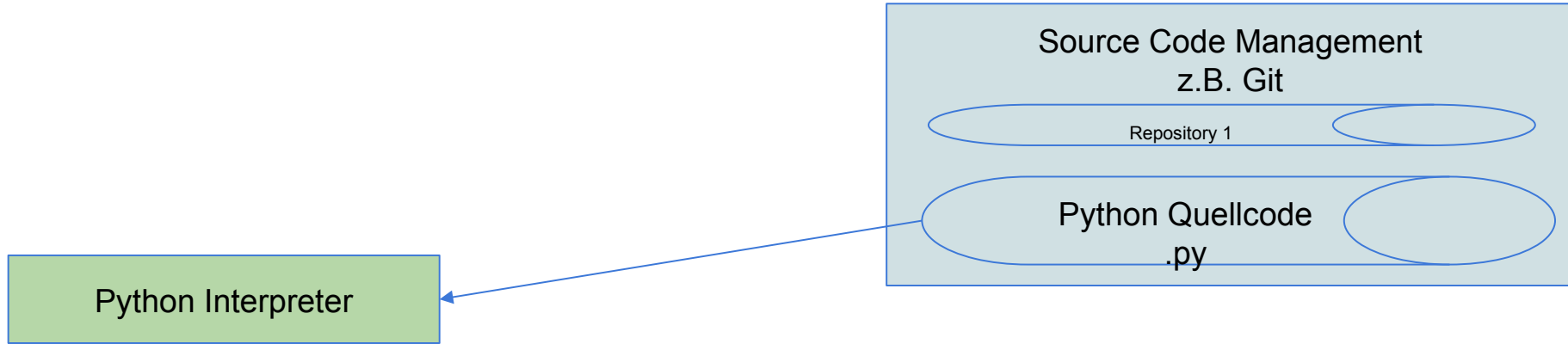
- Name
- Rolle im Unternehmen
- Themenbezogene Vorkenntnisse
- Aktuelle Problemstellung
- Konkrete individuelle Zielsetzung

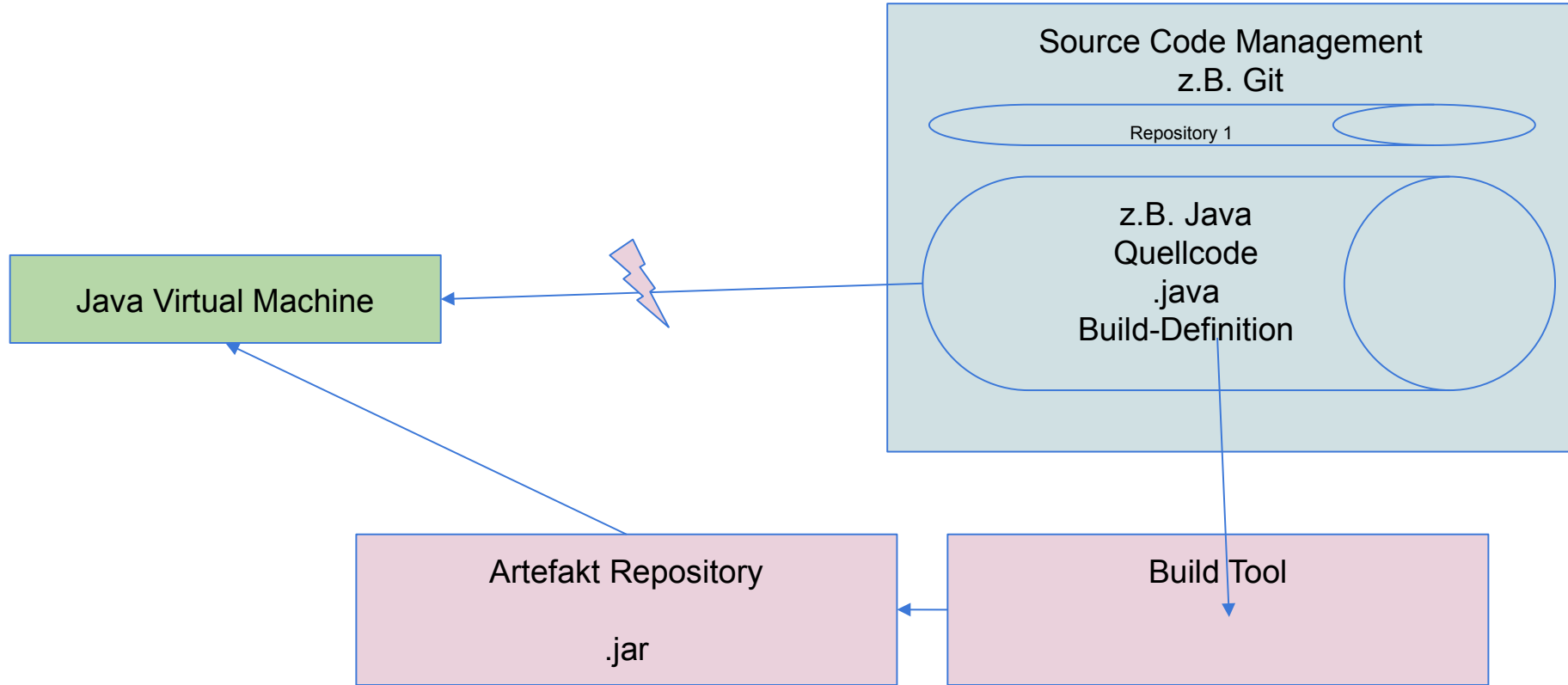


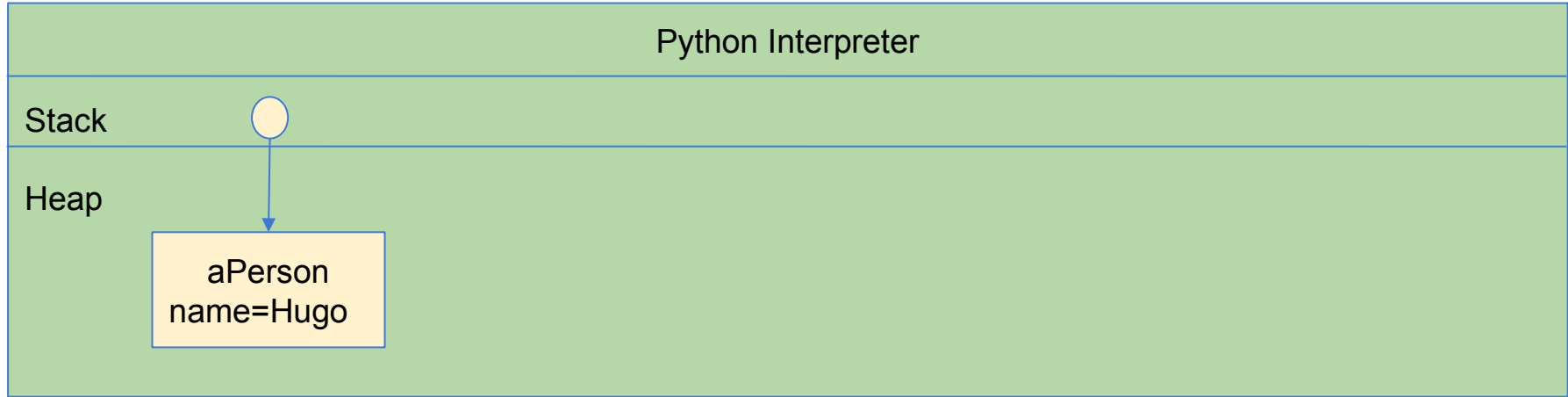
Überblick

- Python ist eine "ganz normale" Programmiersprache
 - Variable, Operatoren, Kontrollstrukturen, Funktionen, ...
 - Syntaktische Besonderheiten sind aber schon vorhanden...
 - Einrückungen sind syntaktisch wichtig
 - "Syntactic Sugar" = elegante Programmierung
 - Comprehension
 - Lambda-Funktion
- Python ist in der Verbreitung aktuell
 - Nummer 1, besser: Top 3
 - Damit haben wir eine große Community mit Ressourcen wie Tutorials, Dokumentationen, Best Practices

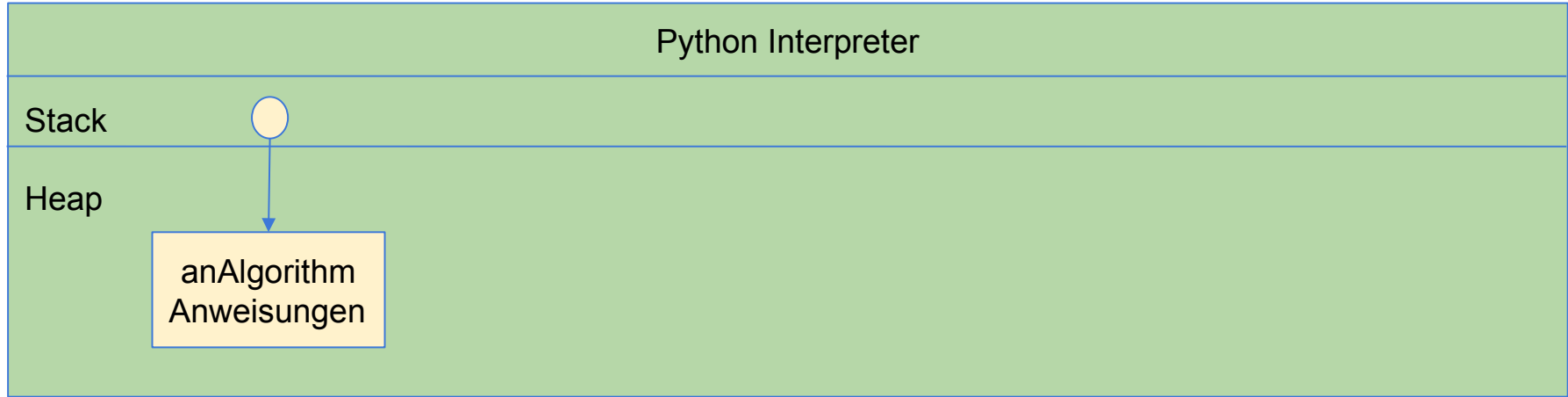








- + Stack ist die Domäne der Programmierung
 - + Variablen
- + Im Heap werden Objekte angelegt und intern verwaltet (insbesondere Garbage Collection)
- + Variablen sind Referenzen auf Objekte



+ Algorithmen = Funktionen sind auch Objekte

- Untypisiert
 - `data = 42`
 - `data` ist damit die Zahl mit dem Wert 42
 - `data = 'Hugo'`
 - `data` ist dann eine Zeichenkette mit dem Wert 'Hugo'
- Offen bedeutet, dass die Struktur / das Datenschema eines Objektes zur Laufzeit veränderbar ist

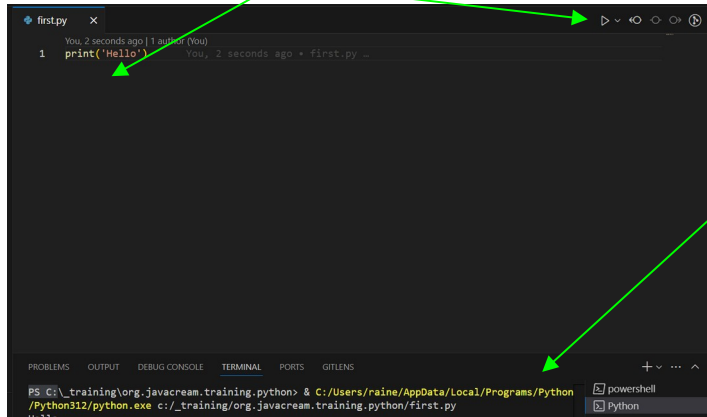
Hinweise

- + Sowohl untypisiert als auch offen wird in der Developer-Community kritisch diskutiert
- + AddOn “typing” führt sogenannte Type Hints ein, `data:int = 42`
- + Programmatisch können Objekte “eingefroren” werden und sind damit statisch fixiert

First Contact

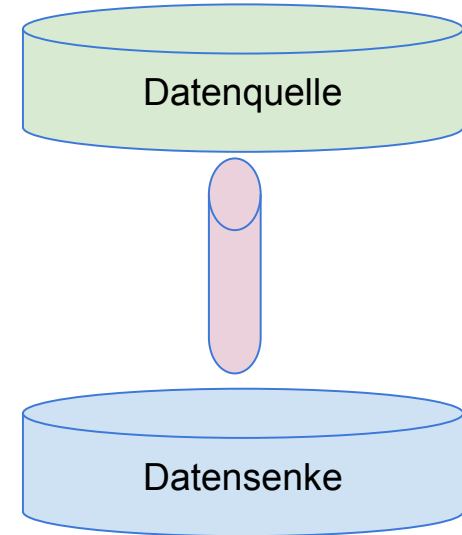
- Visual Studio Code mit Python Extension
 - eine integrierte Entwicklungsumgebung
 - Dateiexplorer
 - Terminal-Fenster
 - Editor
 - Syntax Highlighting
 - Automatische Prüfung (~ Rechtschreibprüfung)
 - Code Assist mit Vorschlägen “was ist als nächstes sinnvoll?”
 - Integration von KI (Copilot, ChatGPT, ...)
 - **Im aktuellen Stadium des Lernens von Python jedoch viel zu umfangreich**
 - Komfortables Starten des Programms

- Check
 - Visual Studio Code installiert?
 - mit den Python Extensions?
 - Öffnen eines Folders
 - Anlegen einer first.py
 - Starten des Programms mit Ausgabe im Terminal

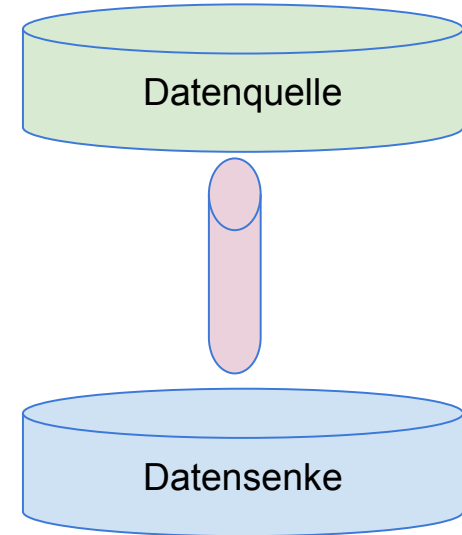


The screenshot shows the Visual Studio Code interface. At the top, a file named 'first.py' is open, containing the code `print('Hello')`. Below the editor, the TERMINAL panel is active, showing the command `PS C:\_training\org.javacream.training.python> & C:/Users/raine/AppData/Local/Programs/Python/Python312/python.exe c:/_training/org.javacream.training.python/first.py` and the output `Hello`. Green arrows point from the list items to the corresponding elements in the screenshot: one from 'first.py' to the file tab, one from 'print('Hello')' to the code line, and one from 'Starten des Programms' to the terminal command.

- Daten einlesen
- Datenverarbeitung
 - Prüfung / Validierung
 - Bereinigung
 - Filterung
 - Transformieren
 - Berechnen / Aggregieren
- Schreiben der Ergebnisse

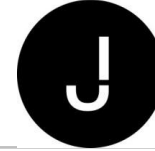


- Daten einlesen **durch Variablen**
- Datenverarbeitung
 - Prüfung / Validierung
 - Bereinigung
 - Filterung
 - Transformieren
 - Berechnen **mit Operatoren**/ Aggregieren
- Schreiben der Ergebnisse **auf die Konsole**



- <https://github.com/Javacream/org.javacream.training.python/blob/72802877eb6524a7b352f33087efe8f73d33d29e/bmi.py>

- Syntaxfehler
 - In VSC mit relativ hoher Wahrscheinlichkeit Unterringelung
 - Das Programm läuft gar nicht, es erfolgt sofort eine Fehlerausgabe
- Laufzeitfehler
 - VSC kann eventuell solche Fehler erkennen
 - NameError, häufig ein Vertippen bei Variablennamen wird gut erkannt
 - Statt 74.1 74,1 (falsche Trenner) -> ValueError mit kryptischer Fehlermeldung und Hinweis nimmt die falsche Zeilennummer
- Logische Fehler
 - z.B. ein Fehler in der Berechnungsformel
 - Hier hilft nur Testen der Anwendung und ein Code Review



- Programmieren mit Copilot / ChatGPT
 - Vorsicht: Diese Präsentation kann dazu führen, dass Sie den Kurs verlassen, weil “der Chat doch schon alles kann”

- Visual Studio Code mit Python Extension
 - eine integrierte Entwicklungsumgebung
 - Dateexplorer
 - Terminal-Fenster
 - Editor
 - Syntax Highlighting
 - Automatische Prüfung (~ Rechtschreibprüfung)
 - Code Assist mit Vorschlägen "was ist als nächstes sinnvoll?"
 - Kostenpflichtig ist die Integration von KI (Copilot, ChatGPT, ...)
 - Im Stadium des Lernens von Python jedoch viel zu umfangreich
 - Komfortables Starten des Programms
 - Debugger
 - Bleibt an den Haltepunkten stehen
 - und visualisiert alle aktuellen Variablen

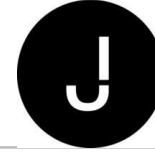
- Werte eines Programms werden durch “Literele” definiert
 - 'Meier', "Meier" -> Zeichenketten
 - 183 -> Ganzzahl
 - 74.1 -> Fließkommazahl
- Hinweis
 - Es gibt noch ein paar weitere Literale -> später
- Typinformation einer Variable wird mit der type-Funktion bekommen
 - `print(type(name))` -> `<class 'str'>`

Hinweis

- + `print` und `type` sind sogenannte BuiltIn-Funktionen in Python, die immer verfügbar sind
- + Insgesamt existieren etwa 60, davon nutzen im Seminar etwa 10

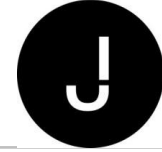
- Erweitern Sie das BMI-Programm
 - Name, Größe, Gewicht als Benutzereingaben
 - Ausgabe
 - <name> hat einen BMI von <bmi mit 2 Nachkommastellen>

- Python-Dokumentation
 - <https://docs.python.org/3/>
 - Sehr technisch, im aktuellen Stand des Seminars unbrauchbar
- Tutorial
 - <https://www.w3schools.com/python/>



Kontrollstrukturen

- Kontrollstrukturen für eine bedingte Programmausführung
 - if - elif - else
 - ~~switch-match case~~ -> später, bzw. Dokumentation
- Schleifen
 - ~~for (index = 0; index < 42; index = index + 1)~~
 - while
 - ~~do while~~
 - for element in container
 - später, dazu brauchen wir Datencontainer
- Programmausführung unter Behandlung von Fehlern
 - try - except
 - Detailliert erst nach OOP-Einführung, aber pragmatisch “bald”



- https://www.w3schools.com/python/python_booleans.asp
- https://www.w3schools.com/python/python_operators.asp

- Nicht (!) blockorientiert
 - if number < 9{
 - Anweisungen für number ist kleiner als 9
 - }
 - else {
 - Anweisungen für number größer gleich 9
 - }

- Nicht (!) blockorientiert
 - if number < 9
 - {
 - Anweisungen für number ist kleiner als 9
 - }
 - else
 - {
 - Anweisungen für number größer gleich 9
 - }

- Nicht (!) blockorientiert
 - `if number < 9{Anweisungen für number ist kleiner als 9}`
 - `else`
 - `{Anweisungen für number größer gleich 9`
 - `}`

Immer noch korrekt, aber brutal
schlecht lesbar -> "Format Code"

- `if number < 9:`
 - Anweisungen für number ist kleiner als 9
- `else:`
 - Anweisungen für number größer gleich 9

Python arbeitet nicht so, sondern nimmt die Einrückungen, die eh zur Lesbarkeit notwendig sind, als Block-Kriterium

- Erweitern Sie das bmi-Beispiel
 - um eine Endlosschleife, die nach jeder Berechnung eine Abfrage (again? (y|n)) durchführt und gegebenenfalls die Schleife abbricht
 - Hinweis
 - `continue`
 - Bricht den aktuellen Schleifendurchlauf ab
 - `break`
 - Bricht die Schleife ab
 - Fehlerbehandlung / Validierung bei den Eingaben von Height und Weight
 - Falls Height < 30 oder größer als 250, Weight < 3 oder > 333
 - Falls Fehler kommt -> Direkt die Frage nach again?...



- DRY
 - Don't repeat yourself
 - = Vermeidung von redundanter Information
- Vermeide unspezifische Konstanten
 - Statt dessen: Sprechend benannte Variablen einführen

- Fehlerbehandlung kann nicht in allen Situationen durch vorherige Prüfung erreicht werden
- "Zahl oder nicht-Zahl" kann nicht so einfach validiert werden!
 - Etwas unsauber
 - Idee
 - Der Algorithmus = die Funktion, die mit einem Parameter arbeitet, validiert diesen Parameter intern

Datencontainer = Collections

■ Allgemein

- Datencontainer gruppieren Informationen, die “irgendwie” zusammengehören
- Über die Elemente des Containers kann mit ‘for in’ iteriert werden
- Die Länge eines Datencontainers = die Anzahl der enthaltenen Elemente ist bestimmbar

- Listen mit Index-Zugriff
 - `todos = ['eat', 'sleep']`
 - `todos = list()`
- Feste Aufzählung mit Tuple
 - z.B. Wochentage, Jahreszeiten
 - `seasons = ('winter', 'spring', 'summer', 'autumn')`
 - `seasons = tuple(['winter', 'spring', 'summer', 'autumn'])`
- Intervalle, also ein Bereich von Zahlen von ... bis
 - `r = range(0, 10)`

Elementzugriff

- + beginnt bei 0
- + Index out of Range
- + Negative Indizes, -1 ist das letzte Element

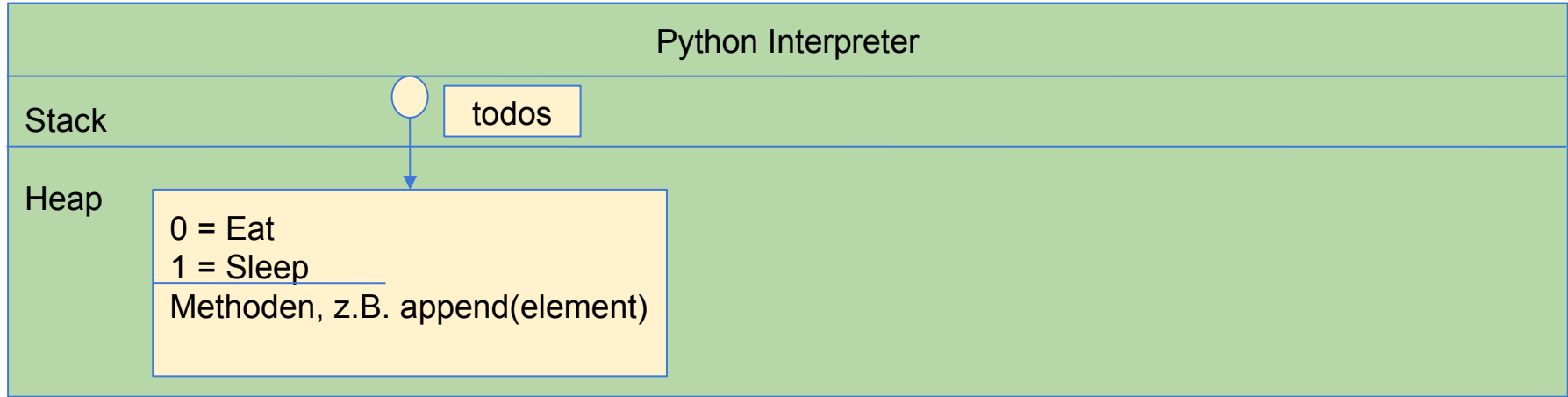
Index-Zugriff,
`seasons[1]`

Index-Zugriff,
`todos[1]`

Index-Zugriff,
`r[1]`

- Menge (im mathematischen Sinn)
 - Duplikate werden identifiziert
 - Vereinigungsmenge (union) bzw. Schnittmenge (intersect)
 - `fruits = {'banana', 'apple', 'orange', 'apple'}`
 - `fruits = set(['banana', 'apple', 'orange', 'apple'])`
- Dictionaries mit key-value-Paaren
 - `postal_codes = {'81371': 'München', '70567': 'Stuttgart'}`
 - `postal_codes = dict()`

Zugriff über key,
`postal_codes['81373']`



- + Zugriff über den Index: `[index]`
- + Zugriff über den `."`-Operator, der im Wesentlichen dem Interpreter sagt 'gehe zu der links angegebenen Referenz und mache das, was rechts neben dem Punkt steht'